

Visible [toolbar]

Shows or hides the selected toolbar in the *Toolbox*.

SimTalk

Visible [SimTalk] - Toolbar

SimTalk Access to the Toolbox

_SimTalk Access to the Toolbox

Remarks

The **Toolbox** provides:

- The *methods* and *attributes* listed in the table of contents to the left.
- The **Methods of All Objects**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.

The screenshot shows the .Models.Model.Station window with the 'deleteAttr' method selected. The method signature is: `(NameOfUserDefinedAttribute:string) -> boolean`. The parameters and return type are highlighted with red boxes.

Name	Value	Inherited	Watchable	Signature
contentsList	[]			((Contents:table)) -> any
createAttr				(NameOfUserDefinedAttribute:string, DataType:string) -> boolean
createIcon				(IconName:string, Width:integer, Height:integer) -> integer
deleteAttr				(NameOfUserDefinedAttribute:string) -> boolean

Name of the method: deleteAttr
 Identifier and data type of the parameter you enter: NameOfUserDefinedAttribute:string
 Data type of the return value: boolean

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, *:= false* in the example above.

If the *method* has a return value, the signature shows its data type after the arrow *->*, *→ boolean* in the example above.

addObject [SimTalk] - Toolbar

Adds the icon of an object to the toolbar designated by <Path>.

Type

Method

Syntax

```
<Path>.addObject(Icon:object[, Position:integer]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Icon* of data type *object* designates the entire path to the object.
- The optional parameter *Position* of data type *integer* designates the position of the icon/button on the

toolbar. The position 01 is to the right of the **Select** button .

Return Value

The return value has the data type *boolean*.

Example

```
.MaterialFlow.Toolbar.addObject(.Materialflow.MyStation,08)  
// adds the icon of the object MyStation to the toolbar Material Flow  
// and positions it to the right of the object Station
```

See also

[Add Objects to the Toolbox or Delete Them from It](#)

delObject [SimTalk]

Deletes the icon of an object from the toolbar designated by <Path>.

Type

Method

Syntax

```
<Path>.delObject(Icon:object) → integer
```

Parameter

The parameter *Icon* of data type *object* designates the entire path to the object.

Return Value

The return value has the data type *integer*.

It is the position of the button, if the object did exist on the toolbar.

It is 0, if it did not exist.

Example

```
.MaterialFlow.Toolbar.delObject(.Materialflow.MyStation)  
// deletes the icon of the object MyStation from the toolbar Material Flow
```

See also

[Remove from Toolbar](#)

[Add Objects to the Toolbox or Delete Them from It](#)

redraw [SimTalk] - Toolbar

Redraws the *Toolbar* designated by <Toolbar-Path> if it is displayed.

Remarks

redraw is useful if you want to modify the *Toolbar* icon of a class, for example with the method [setPixel \[SimTalk\]](#), and if you want to immediately see this change.

Type

Method

Syntax

```
<Toolbar-Path>.redraw
```

Example

```
var newFolder : object := basis.createFolder
var newStation: object := .MaterialFlow.Station.derive(newFolder)

.MaterialFlow.Toolbar.addObject(newStation)
updateGUI
sleep(1, false)

newStation.createIcon("ToolboxIcon", 36, 36)
newStation.CurrIconNo := 0
for var y := 1 to 36
  for var x := 1 to 36
    newStation.setPixel(x, y, makeRGBValue(x*7, 0, y*7))
  next
next

.MaterialFlow.Toolbar.redraw
```

Visible [SimTalk] - Toolbar

Hides (`false`) or shows (`true`) the toolbar designated by `<Path>` in the *Toolbox*.

Type

Attribute

Syntax

```
<Path>.Visible:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
// hides the toolbar Materialflow in the Toolbox
.Materialflow.Toolbar.Visible := false
```

See also

[Visible \[toolbar\]](#)

SimTalk Access to the Folder in the Class Library

_SimTalk Access to the Folder in the Class Library

The *Folder* in the [Class Library](#) provides:

- The *attributes*, *read-only attributes*, and *methods* listed in the table of contents to the left.
- The [Methods of All Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.

Name	Value	Inherited	Watchable	Signature
contentsList	[]			([Contents:table]) -> any
createAttr				(NameOfUserDefinedAttribute:string, DataType:string) -> boolean
createIcon				([IconName:string, Width:integer, Height:integer]) -> integer
deleteAttr				(NameOfUserDefinedAttribute:string) -> boolean

Name of
the method

Identifier and data type of
the parameter you enter

Data type of the
return value

Name	Value	Inherited	Watchable	Signature
moveToFolder				(Folder:object) -> object
MU	VOID			([Index:integer:=1]) -> object
MUPart	VOID			([Index:integer:=1]) -> object

Name of
the method

Identifier and data type of
the parameter you enter

Default
value

Data type of the
return value

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class** [general description].
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance** [general description].

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, `:= false` in the example above.

If the *method* has a return value, the signature shows its data type after the arrow `->`, `→ boolean` in the example above.

createFolder [SimTalk]

Creates a new folder, either at root level under **Basis** in the *Class Library* or in the folder designated by <Folder-Path>.

Remarks

Plant Simulation assigns the name *NewFolder*, *NewFolder1*, to that folder. You can assign a name of your choice by modifying the return value.

Type

Method

Syntax

```
.createFolder → object  
<Folder-Path>.createFolder → object
```

Return Value

The return value has the data type *object*.

Examples

```
// creates a new folder in the folder MaterialFlow and renames it to  
MyFolder  
var obj : object := .MaterialFlow.createFolder  
obj.Name := "MyFolder"
```

```
.createFolder           // creates a folder in the class library  
.MaterialFlow.createFolder // creates a folder in the folder MaterialFlow
```

See also

[New \[Folder, Frame, Toolbar\]](#)

createToolbar [SimTalk]

Creates a new toolbar, either at root level under **Basis** in the *Class Library* or in the folder designated by <Folder-Path>.

Remarks

Plant Simulation assigns the name *Toolbar*, *Toolbar1*, etc. You can assign a name of your choice by modifying the return value.

Type

Method

Syntax

```
.createToolbar → object
<Folder-Path>.createToolbar → object
```

Return Value

The return value has the data type *object*.

Examples

```
// creates a new toolbar in the folder MaterialFlow
// and renames it to MyMaterialfFlowToolbar
var obj: object := .MaterialFlow.createToolbar
obj.Name := "MyMaterialfFlowToolbar"
```

```
.createToolbar           // creates a toolbar in the class library
.MaterialFlow.createToolbar // creates a toolbar in the folder
MaterialFlow
```

See also

[New \[Folder, Frame, Toolbar\]](#)

getLibraryInfo [SimTalk]

Returns if the folder designated by <Folder-Path> is a library (true) or not (false).

Remarks

If the folder is a library, *Plant Simulation* assigns the library information to the local variables of data type *string*, which you specify.

Type

Method

Syntax

```
<Folder-Path>.getLibraryInfo([byref Name:string, byref Version:string,  
byref Description:string, byref FilePathToLibrary:string, byRef  
IsProhibited:boolean, byRef AlternativePaths:string]) → boolean
```

Parameters

You can call the library with one up to seven optional parameters.

- The optional local variable *Name* of data type *string* to which the value is assigned, designates the name of the library.
- The optional local variable *Version* of data type *string* to which the value is assigned, designates the version of the library.
- The optional local variable *Description* of data type *string* to which the value is assigned, designates the description of the library.
- The optional local variable *FilePathToLibrary* of data type *string* to which the value is assigned, designates the file path of the library file (.pslib). This is useful when you want to load data from a file, which is located in the same directory as the .pslib file.
- The optional local variable *IsProhibited* of data type *boolean* to which the value is assigned, designates if the methods of the library do not have access to the computer (true) or do have access to the computer (false).
- The optional local variable *AlternativePaths* of data type *string* to which the value is assigned, designates the alternative paths of the library. If more than one path exists, the paths are separated by a line break (**strAscii(10)**).

Return Value

The return value has the data type *boolean*.

Example

```
.Tools.MyLibrary.getLibraryInfo(MyLibraryName, MyLibraryVersion)
```

SimTalk

[setLibraryInfo \[SimTalk\]](#)

[getLibraryVersionFromFile \[SimTalk\]](#)

See also

[Make Library](#)

[Prohibit Access to the Computer \[Make Library\]](#)

HiddenWhenInLockedFolder [SimTalk]

Sets if the folder designated by <Folder-Path> will not be shown in the *Class Library* when it is located within a locked folder (`true`) or if it will be shown (`false`).

Type

Attribute

Syntax

```
<Folder-Path>.HiddenWhenInLockedFolder : boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
.Models.MyFolder.MyUser.HiddenWhenInLockedFolder := true
```

SimTalk

lockFolder [SimTalk]

IsLocked [SimTalk]

Returns if the folder designated by <Folder-Path> is a locked library (true) or not (false).

Type

Read-only attribute

Syntax

```
<Folder-Path>.IsLocked -> boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
print .MyFolder.IsLocked
```

See also

[Lock Library](#)

loadObjectAs [SimTalk]

Loads an object, which you saved to a file—with the extension .psobj—into the folder designated by <Folder-Path>.

Remarks

In most cases this will be *Class Library* itself, designated by . or by basis.

Type

Method

Syntax

```
<Folder-Path>.loadObjectAs(Object:string[, NewName:string,  
Automerge:boolean:=false, ReplaceExistingClass:boolean:=false,  
Password:string]) → object
```

Parameters

You can specify the following parameters:

- The parameter *Object* of data type *string* designates the object.
- The parameter *NewName* of data type *string* designates the new name of the object if you would like to assign a different name. To define the *boolean* parameters, but keep the existing name, specify an empty string, i.e., a string of length zero (""), as the parameter *string*.

The loaded object may contain classes, which already exist in the *Class Library*. These classes have to be merged.

- The optional parameter *AutoMerge* of data type *boolean* sets if **AutoMerge** is active (*true*) or if *Plant Simulation* opens a dialog in which you manually select how it merges objects (*false*). In addition, *Plant Simulation* does not create a **Merge Report**, when **AutoMerge** is active.

If **AutoMerge** is *true*, the optional parameter of data type *boolean* sets if the newly loaded classes replace the existing classes (*true*) or if the existing classes in your *Class Library* will be retained (*false*). To retain the existing objects in your simulation model and to discard duplicate objects contained in the loaded model, use **Load Object** and then select **Replace the Loaded Class With the One in the Class Library**.

Default Value of the Parameter

The default value is *false*.

- The optional parameter *ReplaceExistingClass* of data type *boolean* determines whether the newly loaded classes will replace existing ones (*true*) or if the existing classes in your *Class Library* will be preserved (*false*), when **AutoMerge** is *true*.

To preserve the existing objects in your simulation model, and to discard duplicate objects that are contained in the model you load, you will use **Load Object**, and then select to **Replace the Loaded Class with the One in the Class Library**.

Selecting the context menu command **Save/Load > Load Object** in the *Class Library* accomplishes the same as setting the parameter *boolean* to `false`. Selecting the context menu command **Save/Load > Update Class Library** accomplishes the same as setting the parameter *boolean* to `true`.

Note

This parameter does not have any effect when loading a library because *Plant Simulation* determines according to the version number if the library will be updated or not.

Default Value of the Parameter

The default value is `false`.

- The optional parameter *Password* of data type *string* sets the password which is going to be used to load an encrypted object file.

Return Value

The return value has the data type *object*.

Example

```
// loads the object named protocol and automatically merges it
.loadObjectAs("C:\Users\MyName\protocol.psobj", "Protocol", true, false)
Protocol.CurrIcon := "operational"
```

SimTalk

autoexecLoadObj [SimTalk]

lockFolder [SimTalk]

Locks the folder designated by <Folder-Path> with a password.

Remarks

Plant Simulation prevents objects, which are located inside of a locked folder, from being opened interactively.

To unlock a library folder, clear the check box **Lock Library** in the dialog **Manage Class Library** and enter the password. Alternatively you can hold down **Shift** while clicking the **OK** button in the dialog of the error message that *Plant Simulation* showed when you tried to open the object. Then *Plant Simulation* prompts you for the password of the locked folder. This temporarily suppresses the lock of the designated library folder. When you save your model and open it again, *Plant Simulation* does not suppress the lock any more.

You can, for example, use the method *lockFolder* to prevent that libraries are inadvertently changed. Changes via information flow are still possible though.

Plant Simulation prevents only the objects, which are located within the locked folder, from being opened interactively, but **not** instances of those objects which have been inserted into *Frames* outside of the locked folder.

Type

Method

Syntax

```
<Folder-Path>.lockFolder(Lock:boolean/void, OldPassword:string[,  
NewPassword:string/void, EncryptMethods:boolean])
```

Parameters

You can specify the following parameters:

- The parameter *Lock* of data type *boolean* sets if the folder and all of its sub-folders will be locked (*true*), or unlocked (*false*), or if the lock will be removed (*void*).
The value *false* removes the lock that was set by a surrounding folder. The value *void* removes a lock and reactivates a possibly existing lock of the surrounding folder.
- The parameter *OldPassword* of data type *string* designates the old password.
When you call the method *lockFolder* for an already locked folder, you have to enter the password for this parameter. If you enter the wrong password, the method *lockFolder* fails. If the folder has not been locked yet, the parameter *OldPassword* will be ignored.

- The optional parameter *NewPassword* of data type *string* designates the new password.

You only have to specify the parameter *NewPassword* if the parameter *Lock* is set to `true`. Specify an empty string `""` to not set a password. Otherwise you can only remove the lock by entering the correct password.

- The optional parameter *EncryptMethods* sets if all methods, which are located within the folder and are not encrypted, are to be encrypted (`true`) or not (`false`). If you do not specify the parameter, the methods will not be encrypted. If the parameter *Lock* is not `true`, the parameter *EncryptMethods* will be ignored.

Example

```
.Models.MyFolder.MyUser.lockFolder(true, "My Old Password", "MyNewPassword")
```

```
.Models.MyFolder.MyUser.lockFolder(true, "My Old Password", "") -- no new password
```

SimTalk

[HiddenWhenInLockedFolder \[SimTalk\]](#)

[IsLocked \[SimTalk\]](#)

See also

[Lock Library](#)

node [SimTalk] - folder

Calls an object within the folder designated by <Folder-Path>.

Remarks

Plant Simulation numbers the objects within folders in your model in the order in which you insert them.

Type

Method

Syntax

```
<Folder - Path>.node(ObjectNumberOrName:integer) → object
```

Parameter

The parameter *ObjectNumber* of data type *integer* designates the number of the object.

Return Value

The return value has the data type *object*.

Example

```
print .Models.MnEnginePlant.node(5)  
// returns the path and the name of the object designated by node,  
// for example .Models.EnginePlant.Track
```

See also

[numNodes \[SimTalk\] - folder](#)

numNodes [SimTalk] - folder

Returns the number of objects in the folder designated by <Folder-Path>.

Remarks

Each additional folder you insert into the selected folder counts as a single object. *Plant Simulation* does not count the objects contained within the folders.

Type

Method

Syntax

```
<Folder-Path>.numNodes → integer
```

Return Value

The return value has the data type *integer*.

Example

```
print .Models.MyPlant.numNodes
```

SimTalk

node [SimTalk] - folder

pasteClipboard [SimTalk] - class library

Pastes the contents of the clipboard into the folder designated by <Folder-Path>.

Type

Method

Syntax

```
<Folder-Path>.pasteClipboard([TargetTableOfPastedObjects:table])
```

Parameter

The optional parameter *TargetTableOfPastedObjects* of data type *table* sets, if *Plant Simulation* writes all pasted objects to this table after it executed the method. The table has one column of data type *object*.

Examples

```
var tbl : table
pasteClipboard(tbl)
for var i := 1 to tbl.yDim
    print tbl[1,i] // returns all pasted objects
next
```

```
.Materialflow.pasteClipboard(MyPastedObjects)
```

See also

[Paste \[object class\]](#)

setLibraryInfo [SimTalk]

Sets if *Plant Simulation* treats the folder designated by <Folder-Path> as a library, and sets the name and the version of the library.

Type

Method

Syntax

```
<Folder-Path>.setLibraryInfo(Name:string, Version:string[,  
Description:string, AccessProhibited:boolean])
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the **name** of the library.
- The parameter *Version* of data type *string* designates the **version** of the library. You can enter any number of numbers separated by periods.
- The optional parameter *Description* of data type *string* designates the description of the library. If you do not assign this parameter, the current description will not be changed.
- The optional parameter *AccessProhibited* of data type *boolean* sets that the library cannot access the computer (`true`). If you do not specify the parameter, the setting **Prohibit Access to the Computer [Make Library]** will not be changed.

Note

You cannot specify the value `false` as, for security reasons, it is not allowed to grant a library access to the computer via information flow.

SimTalk

getLibraryInfo [SimTalk]

See also

Make Library

setRequiredLicense [SimTalk]

Tells the library folder designated by <Folder-Path> to request a user-defined license.

Remarks

When you load a simulation model or an object, which contains such a library, *Plant Simulation* looks for the corresponding user-defined license in the *Windows Registry*. If it does not find a valid license with the specified feature name, *Plant Simulation* cannot load the simulation model or the object respectively.

The registered license is only valid when the version is greater than or equal to the requested version, when the **Host-ID** of the license matches the **Host-ID** of the computer and when the validity date of the license has not been exceeded, and when the password of the registered license matches the password of the requested license.

By using a user-defined license you can restrict the passing on of simulation models and objects, including libraries.

- To restrict the passing on of a simulation model, call `basis.setRequiredLicense` and then save the model.
- To restrict the passing on of a library, call `setRequiredLicense` for the library folder and then save the library.

When you update a library that requires a user-defined library, the new version of the library also has to request this user-defined license (i.e., the same feature), and the requested version number has to be as least as great as the old library, and the license has to be protected with the same password.

To assign a different password to the newer version of the library, call the method `setRequiredLicense` and specify an empty string "" for the parameter *Feature* and the old password as the parameter *Password*. This de-registers the user-defined license. Then call the method `setRequiredLicense` again and specify the name of the license feature to the parameter *Feature* and the new password to the parameter *Password*, and the old password to the parameter *PreviousPasswordForUpdate*. The library can then not only update versions of the library that use the new password according to the rules described above, but also any version of the library that use the old password, even then when the name of the license feature changes or the new version number is smaller than the version number of the old library.

Type

Method

Syntax

```
<Folder-Path>.setRequiredLicense(Feature:string, Version:string,  
Password:string[, FreeInLicense:string, Comment:string,  
PreviousPasswordForUpdate:string])
```

Parameters

You can specify the following parameters:

- The parameter *Feature* of data type *string* designates the name of the license feature. Specify an empty string "" to remove an existing license provided you specify the correct password.
- The parameter *Version* of data type *string* designates the version number of the license. The registered version has to be greater than or equal to the requested license.

If, for example, version 10 of the feature is registered, you can request version 9.5.0 but not vice versa.

- The parameter *Password* of data type *string* designates a secret password. The password of the registered license has to match the this password so that the registered license is considered valid.
- The optional parameter *FreeInLicense* of data type *string* sets that the user-defined license is not requested when *Plant Simulation* uses the requested license type or a license type of lower value. You can enter "Application", "Educational", "Runtime", "Simulation", "", or "-".

The values have the following meaning:

"-": The license is never free, not even in the *Viewer license*.

"": The license is only free in the *Viewer license*. This is the default setting when you do not enter the parameter.

"Simulation": The license is free in the *Simulation* and in the lower *Viewer license*.

"Runtime": The license is free in the *Runtime license* and in the lower license types, i.e., *Simulation* and *Viewer*.

"Application": The license is free in the *Application license* and in the lower license types, i.e., *Runtime*, *Simulation*, and *Viewer*.

"Educational": The license is free in the *Educational license* and in the lower license types, i.e., *Runtime*, *Simulation*, and *Viewer*.

- The optional parameter *Comment* of data type *string* sets the text which will be attached to the message, which is shown in the message window when no license exists when loading the object or the model. This only applies if you did enter any text.
- The optional parameter *PreviousPasswordForUpdate* of data type *string* designates the previous password for updating the library.

You can generate user-defined licenses with the *SimTalk* function *createLicenseFile*.

Example

```
.Standard.MyLibrary.setRequiredLicense("MyFeature", "9.5.7",  
"MySecretPassword123")
```

[setRequiredLicense \[SimTalk\]](#)

SimTalk

[checkForLicense \[SimTalk\]](#)

[createLicenseFile \[SimTalk\]](#)

ShowToolbar [SimTalk]

Shows (`true`) or hides (`false`) the *Toolbar* of the specified folder in the *Class Library* in the *Toolbox*

Type

Attribute

Syntax

```
<Folder-Path>.ShowToolbar:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
.MaterialFlow.ShowToolbar := false
```

See also

[Show Toolbar](#)

Console

The *Console* shows information about actions, which *Plant Simulation* executes.

Remarks

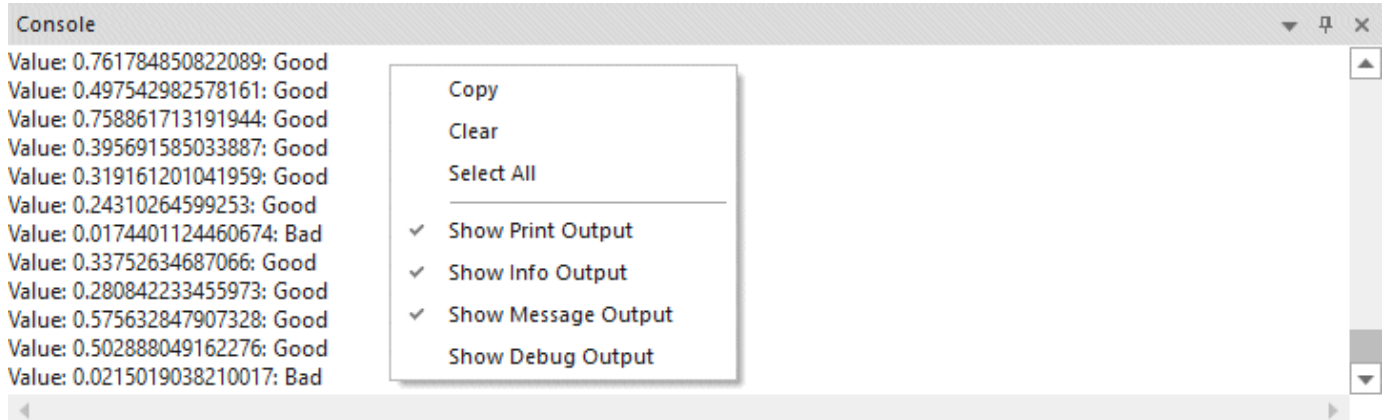
You can select which information the *Console* window shows under **File > Preferences > User Interface > Console Filter**.

To record messages, display the *Console* by clicking  on the **Window** ribbon tab. To not record messages, hide it.

To close the *Console* window, click the X in the title bar of the window. To show it again, click  on the **Window** ribbon tab.

To auto hide the *Console*, click . When you then click in another window, *Plant Simulation* hides the window until you move the mouse over the name of the window again. To deactivate **Auto Hide**, click .

The context menu of the *Console* window provides these commands:



- To copy the contents of the *Console* window, select the text it contains and then select **Copy**.
- To delete the contents of the *Console* window, select **Clear**.
- To select the entire text in the *Console* window, select **Select All**.
- **Show Print Output**

To show output from **print** commands in *Methods*, select this command.

- **Show Info Output**

To show text, which is displayed in info boxes, such as those shown during the **reset** and **init** phases of the model, select this command. In addition, it includes warnings, for example when you use a